

Document number: P0358R1
Date: 2016-06-22
Project: Programming Language C++, Library Evolution Working Group
Reply-to: Tomasz Kamiński <tomaszkam at gmail dot com>

Fixes for not_fn

1. Introduction

In this paper new wording for `std::not_fn` is proposed, that amongst other improvements, provides support of propagation of value category in case of wrapper invocation.

2.. Revision History

2.1. Revision 1

- Updated wording according to LWG review guidance.
- Incorporated numerous motivation section corrections and suggestion from Patrice Roy's review.

3. Motivation and Scope

The main motivation for creation of this paper comes from the realisation that the existing wording for `std::not_fn`, that was recently accepted in the C++17, requires implementation to always perform call on the lvalue of the stored callable.

This effectively requires `std::not_fn` to disregard the reference qualification of nested function. For example in case of the following definitions:

```
struct RvalueCallable
{
    bool operator>() &&;
};

struct LvalueCallable
{
    bool operator>() &;
};

auto rval = std::not_fn(RvalueCallable{});
auto lval = std::not_fn(LvalueCallable{});
```

Both the invocation `rval()` and `std::move(rval)()` are ill-formed, as they will lead to call on the `RvalueCallable&`. In addition the call `std::move(lval)()` is well-formed despite explicit reference qualification of the `LvalueCallable::operator()`, that should prevent invocation on temporaries.

It also important to emphasize that introduction of the value category propagation after standardization of the current wording would be a breaking change. Furthermore, such breakage may be silent in case when the wrapped callable was providing both lvalue and rvalue overloads of the call operator - in case of temporary wrappers, the code will start to invoke rvalue overloads.

4. Wording Discussion

After finding above flaws in the current wording, I (author of this paper, that also happens to be author of the wording) have decided to review it thoughtfully again. This section contains list of a possible problems that I was able to identify.

Note: Wording used for `not_fn` was largely based on existing wording of the `std::bind`, so most of the issue are common to these components.

4.1. Effects of the invocation

In the newest working draft [N4582](#) (C++ Working Draft, 2016-03-19), effects of invocation of the `std::not_fn` created wrapper are defined as follows (20.12.9 [func.not_fn]):

Returns: A forwarding call wrapper `g` such that the expression `g(a1, a2, ..., aN)` is equivalent to `! INVOKE(fd, a1, a2, ..., aN)` (20.12.2).

Where:

- `FD` is the type `decay_t<F>`,
- `fd` is an lvalue of type `FD` constructed from `std::forward<F>(f)`,
- `g` is a forwarding call wrapper created as a result of `not_fn(f)`,

Firstly, we may notice that the definition of `fd` explicitly states that it is **lvalue** reference, which results in the problems described in motivation section.

Secondly, the above wording does not mention how `cv` qualification will affect underlining functor. To be explicit, the wording only describes effects of the invocation on the temporary created as the result of `not_fn` invocation. As a consequence, implementations that will provide only single overload of call operator in form:

```
template<typename... Args>
decltype(auto) operator()(Args&&... args) &&
{ return std::invoke(fd, std::forward<Args>(args)...); }
```

could be considered as standard conforming, despite is questionable usability.

The author believes that the wording should guarantee that for the every invocation `static_cast<G cv ref>(g)(args...)` is equivalent to `static_cast<FD cv ref>(fd)(args...)`, where:

- `G` is type of `g`
- `cv` is either `const` or empty
- `ref` is either `&` or `&&`

In addition, current wording does not clarify if the invocation `std::not_fn(f)(args...)` are equivalent to `!std::invoke(f, args...)` in unevaluated context. Such a guarantee is important for components that conditionally exposes features, depending on the validity of the call expression. Most notable example of such functionality is `std::function<R(Args...)>` that is only constructible from type `F`, that is lvalue callable with `Args...` and has return compatible with `R`.

4.2. Move/copy operations on the wrapper

Existing wording describes requirements on transferability of the created wrapper as follows (20.12.9 [func.not_fn]):

The return type shall satisfy the requirements of `MoveConstructible`. If `FD` satisfies the requirements of `CopyConstructible`, then the return type shall satisfy the requirements of `CopyConstructible`. [*Note*: This implies that `FD` is `MoveConstructible`. — *end note*]

The wording is requiring that the wrapper will expose the same set of the operations as the wrapped callable, however it is not defined how these operations would be defined in terms of corresponding operations on the nested functor type `FD`. As example in case when the `FD` is `CopyConstructible`, conforming implementation of the wrapper could expose only copy constructor (without declaration of move constructor) as every `CopyConstructible` is by definition `MoveConstructible`.

In addition is is not specified if the exposed move operation of the wrapper would have the same exception specification as the corresponding operation of the underlining wrapper. As consequence wrapping a function pointer or simple lambda object into `not_fn`, may disable optimizations that are requiring nothrowing move operations, like using small object optimization in `std::function`.

5. Proposed wording

The proposed wording changes refer to [N4582](#) (C++ Working Draft, 2016-03-19).

Change the section 20.12.9 Function template `not_fn` [func.not_fn] to:

```
template <class F>
    unspecified not_fn(F&& f);
```

Effects: Equivalent to return `call_wrapper(std::forward<F>(f))`, where `call_wrapper` is an exposition only class, defined as follows:

```
class call_wrapper
{
```

```

using FD = decay_t<F>;
explicit call_wrapper(F&& f);

public:
    call_wrapper(call_wrapper&&) = default;
    call_wrapper(call_wrapper const&) = default;

    template<class... Args>
        auto operator()(Args&&...) & -> decltype(!declval<result_of_t<FD&(Args...)>>());

    template<class... Args>
        auto operator()(Args&&...) const& -> decltype(!declval<result_of_t<FD const&(Args...)>>());

    template<class... Args>
        auto operator()(Args&&...) && -> decltype(!declval<result_of_t<FD(Args...)>>());

    template<class... Args>
        auto operator()(Args&&...) const&& -> decltype(!declval<result_of_t<FD const(Args...)>>());

private:
    FD fd;
};

explicit call_wrapper(F&& f);

```

Requires: FD shall satisfy the requirements of `MoveConstructible`. `is_constructible_v<FD, F>` shall be true. fd shall be a callable object ([func.def] 20.12.1).

Effects: Initializes fd from `std::forward<F>(f)`.

Throws: Any exception thrown by construction of fd.

```

template<class... Args> auto operator()(Args&&... args) & -> decltype(!declval<result_of_t<FD&(Args...)>>());
template<class... Args> auto operator()(Args&&... args) const& -> decltype(!declval<result_of_t<FD const&(Args...)>>());

```

Effects: Equivalent to return `!INVOKE(fd, std::forward<Args>(args)...) ([func.require] 20.12.2).`

```

template<class... Args> auto operator()(Args&&... args) && -> decltype(!declval<result_of_t<FD(Args...)>>());
template<class... Args> auto operator()(Args&&... args) const&& -> decltype(!declval<result_of_t<FD const(Args...)>>());

```

Effects: Equivalent to return `!INVOKE(std::move(fd), std::forward<Args>(args)...) ([func.require] 20.12.2).`

7. Acknowledgements

Stephan T. Lavavej suggested numerous corrections to [comprehensive wording](#) presented in the paper.

Patrice Roy provided numerous corrections and improvement for the paper.

Special thanks and recognition goes to Sabre (<http://www.sabre.com>) for supporting the production of this proposal, and for sponsoring author's trip to the Oulu for WG21 meeting.

8. References

1. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4582, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4582.pdf>)